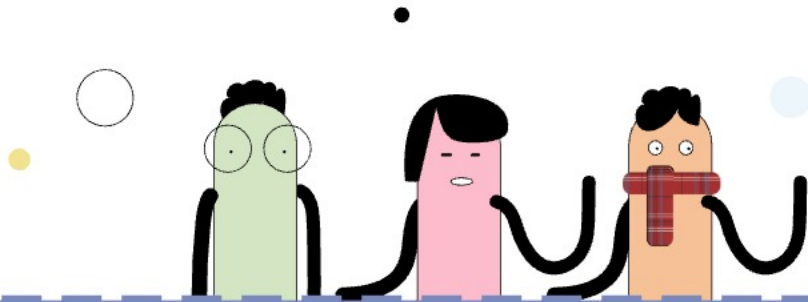




Golang

Advanced

Generic

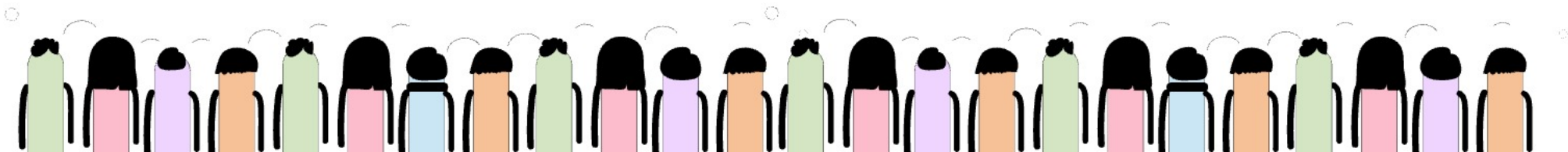
1 You write the same logic again and again

Without generics (what teams actually suffer from):

Go

```
func GetUserByID(id int) (User, error) { ... }  
func GetOrderByID(id int) (Order, error) { ... }  
func GetProductByID(id int) (Product, error) { ... }
```

- 👉 Same logic
- 👉 Different types
- 👉 Code duplication everywhere

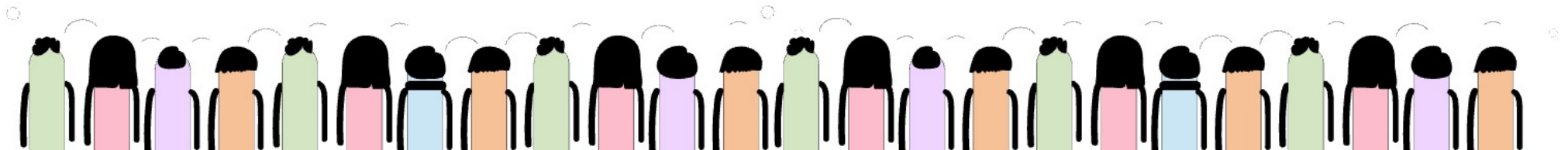


Generic

- You are building **reusable layers**
 - repositories
 - caches
 - API wrappers
- You see **same logic repeated across types**

Avoid when:

- Business logic is specific
- It reduces readability



Reflect

Case 1: Passing VALUE

</> Go

```
val := reflect.ValueOf(u)
```

👉 Internally:

`reflect.Value` → copy of `u`

- ✓ You are working on a **copy**
- ✓ Cannot modify original
- ✓ Safe read-only usage

Case 2: Passing POINTER

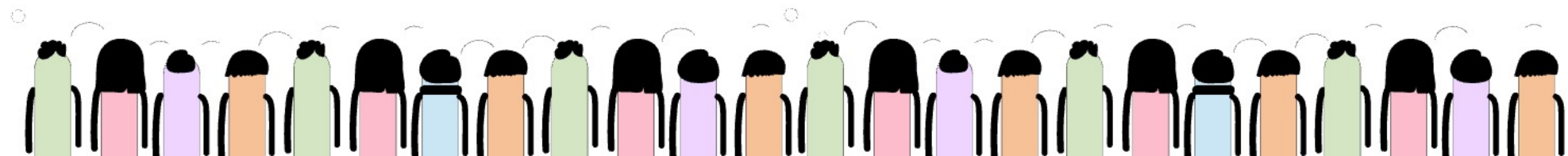
</> Go

```
val := reflect.ValueOf(&u)
```

👉 Internally:

`reflect.Value` → holds address of `u`

- ✓ Now reflection has **access to original memory**
- ✓ You can modify (after `.Elem()`)



Reflect

Line

Meaning

```
reflect.ValueOf(input)
```

Get runtime value

```
Kind() == reflect.Ptr
```

Check if pointer

```
.Elem()
```

Unwrap pointer → actual struct

```
NumField()
```

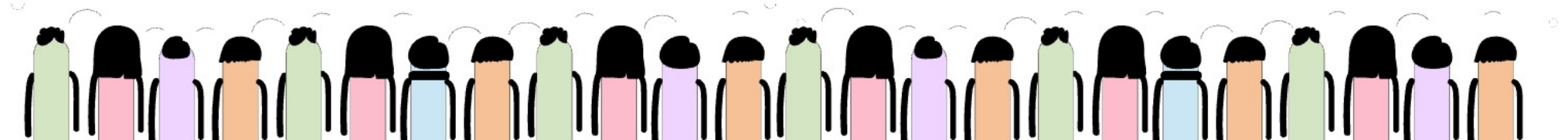
Works ONLY on struct



Reflect

Final Truth (Important)

Concept	Is it Pointer?	What it does
<code>ValueOf</code>	✗ No	Wraps value (pointer OR struct)
<code>Kind()</code>	✗ No	Checks type
<code>.Elem()</code>	✓ YES	Dereferences pointer
<code>NumField()</code>	✗ No	Works on struct only





GoRoutine



Think of a train booking system:

Step flow

Search → Seat availability → Payment → Confirmation

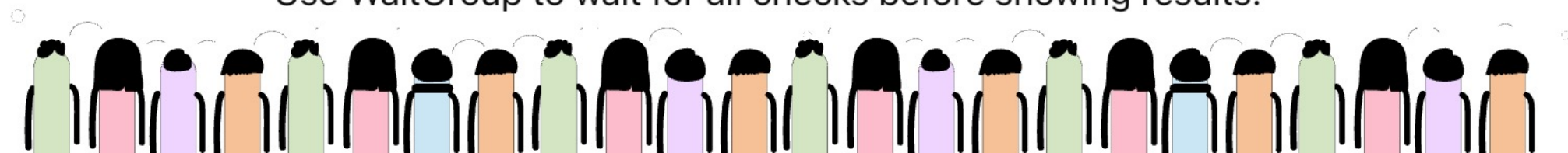
Where each concept fits

WaitGroup

Parallel checks:

- multiple quota checks (general, tatkal, premium)
- multiple coach availability

Use WaitGroup to wait for all checks before showing results.



GoRoutine

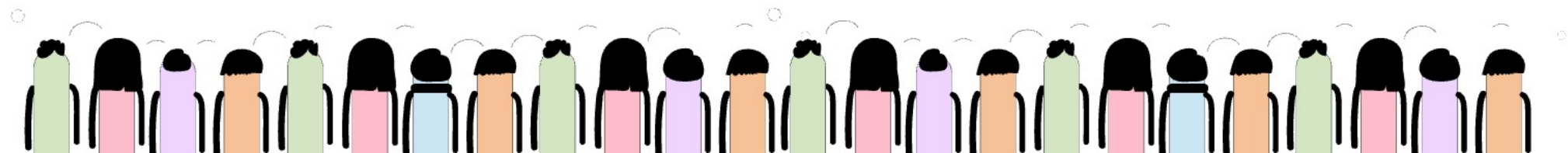
Worker pool

High traffic search:

- thousands of users searching trains
- each search triggers availability checks

Use worker pool to:

- limit DB/API load
- avoid spawning unlimited goroutines



GoRoutine

Context cancellation

User behavior:

- user searches and quickly changes route
- previous request becomes useless

Use context:

- cancel DB queries
- stop seat computation
- release resources immediately



GoRoutine

3) Distributed system style example

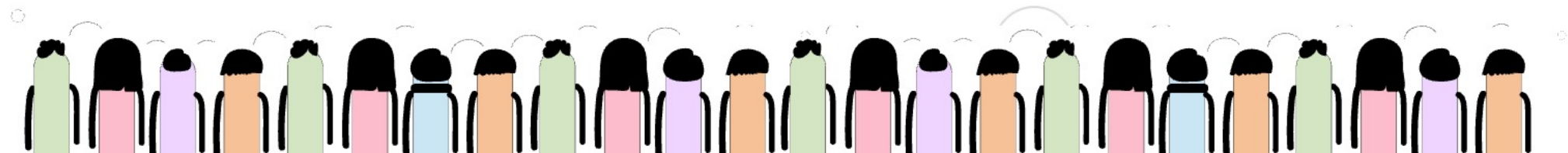
Scenario:

Search service calls multiple downstream services:

- seat service
- pricing service
- quota service

Requirements:

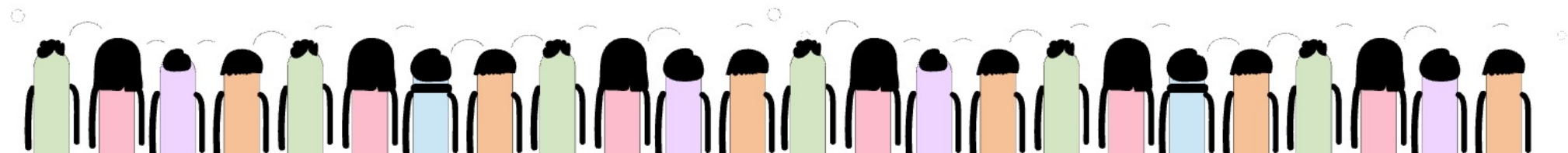
- run calls in parallel
- limit concurrency
- cancel all if timeout occurs
- aggregate results



GoRoutine

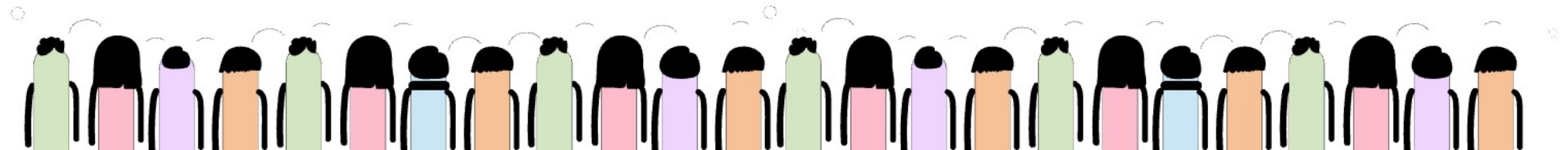
What this code demonstrates

- worker pool controls concurrency
- WaitGroup ensures all workers finish
- context enforces timeout across all services
- parallel service calls
- safe aggregation of results

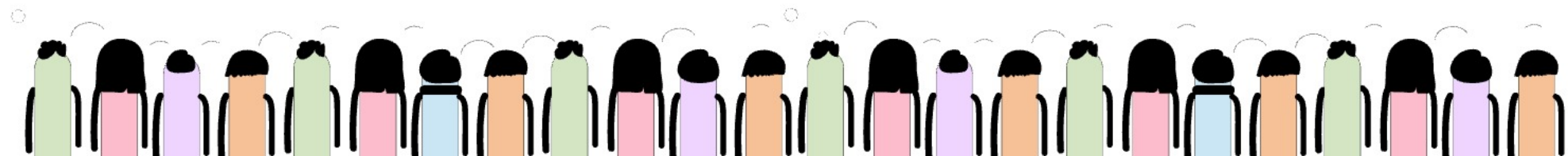
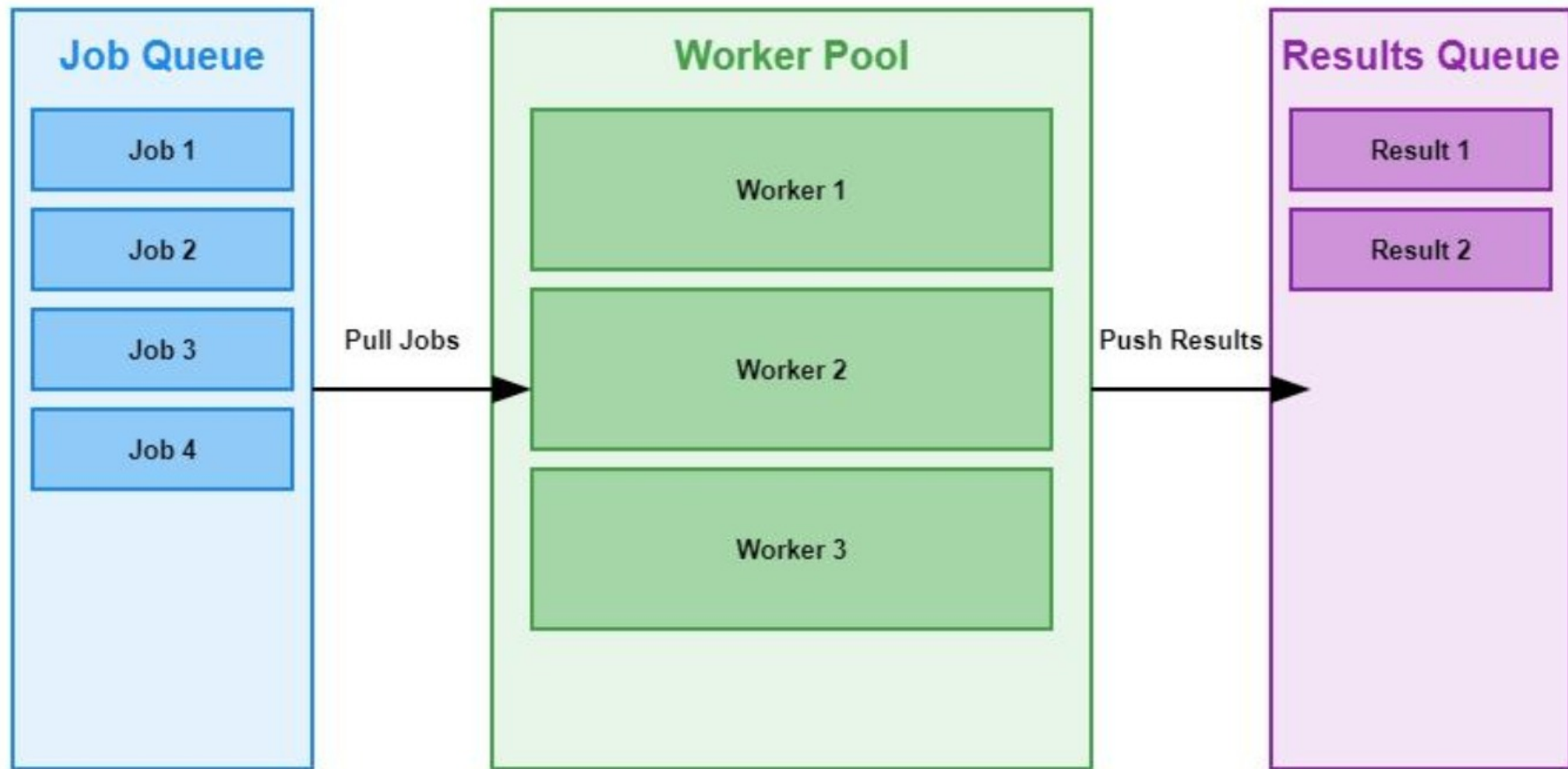


Testing

```
├── go.mod
├── internal/
│   ├── mathutil/
│   │   ├── add.go
│   │   ├── add_test.go
│   │   └── fuzz_test.go
│   ├── http/
│   │   ├── handler.go
│   │   └── handler_test.go
│   └── logging/
│       └── logger.go
└── cmd/
    └── app/
        └── main.go
```



System design



System design

Client / Source



Job Producer



Buffered Queue (Backpressure)



Worker Pool (CPU work)



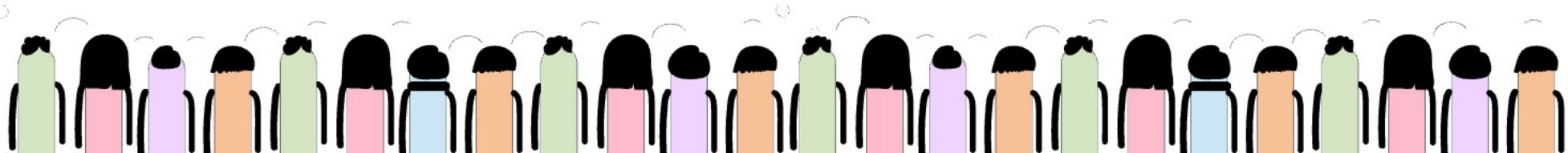
Result Queue



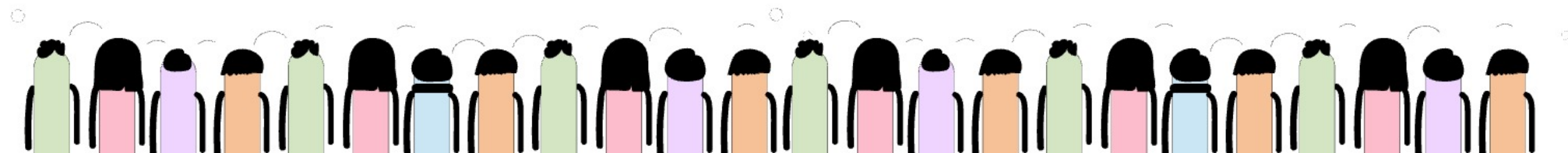
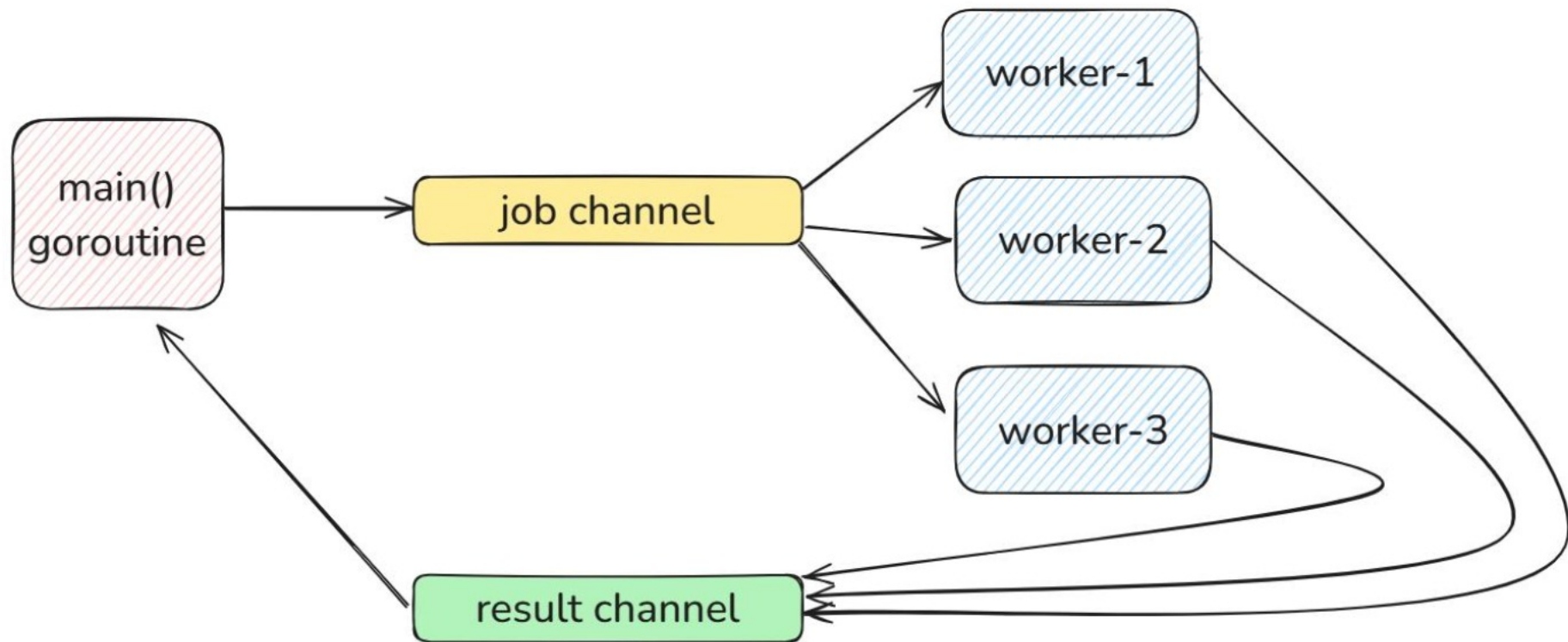
Batch Writers



Database (limited connections)



System design



System Flow

MAIN THREAD

```
|  
|-- start workers (G1...Gn)  
|-- start writers (W1...W4)  
|-- start producer  
|  
↓
```

PRODUCER (goroutine)

```
|  
|-- generate 1M jobs  
|-- push → jobs channel  
|-- close(jobs)
```

WORKERS (goroutines)

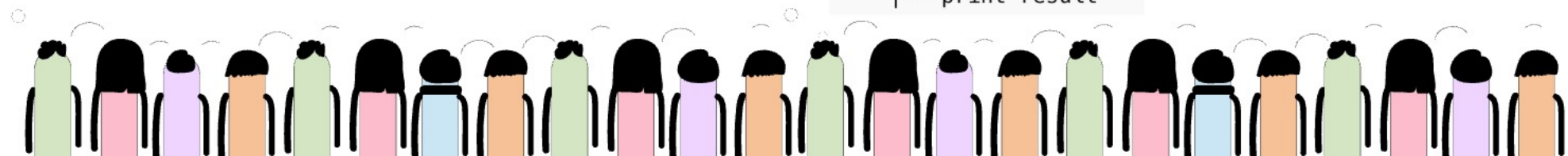
```
|  
|-- read from jobs  
|-- process() CPU work  
|-- push → results channel  
  
↓
```

WRITERS (goroutines)

```
|  
|-- read from results  
|-- batch (500 items)  
|-- writeBatch()  
|-- DB semaphore control  
  
↓
```

MAIN

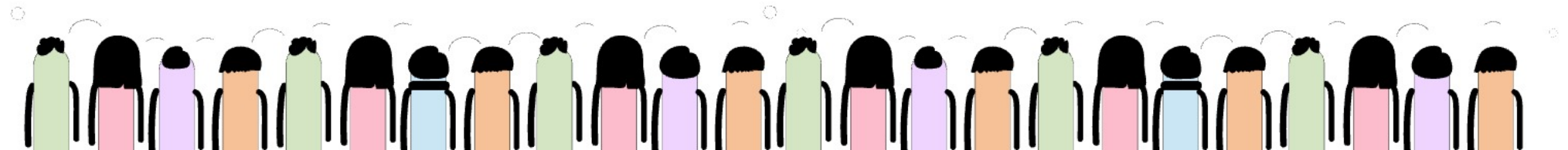
```
|  
|-- wait writerWG  
|-- print result
```



System design

👉 Payment Processing System

- Service1 → sends payment request
- Service2 → processes payment
- Must avoid **duplicate charges**



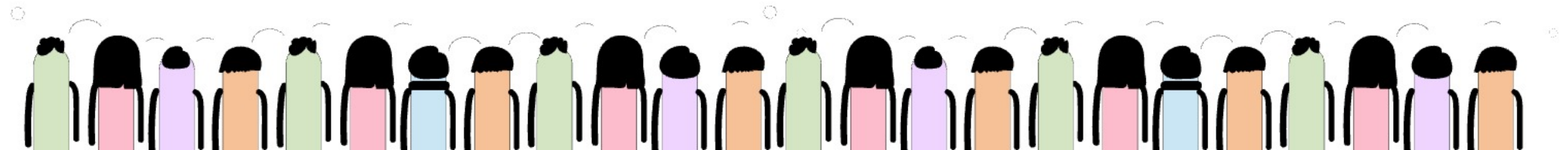
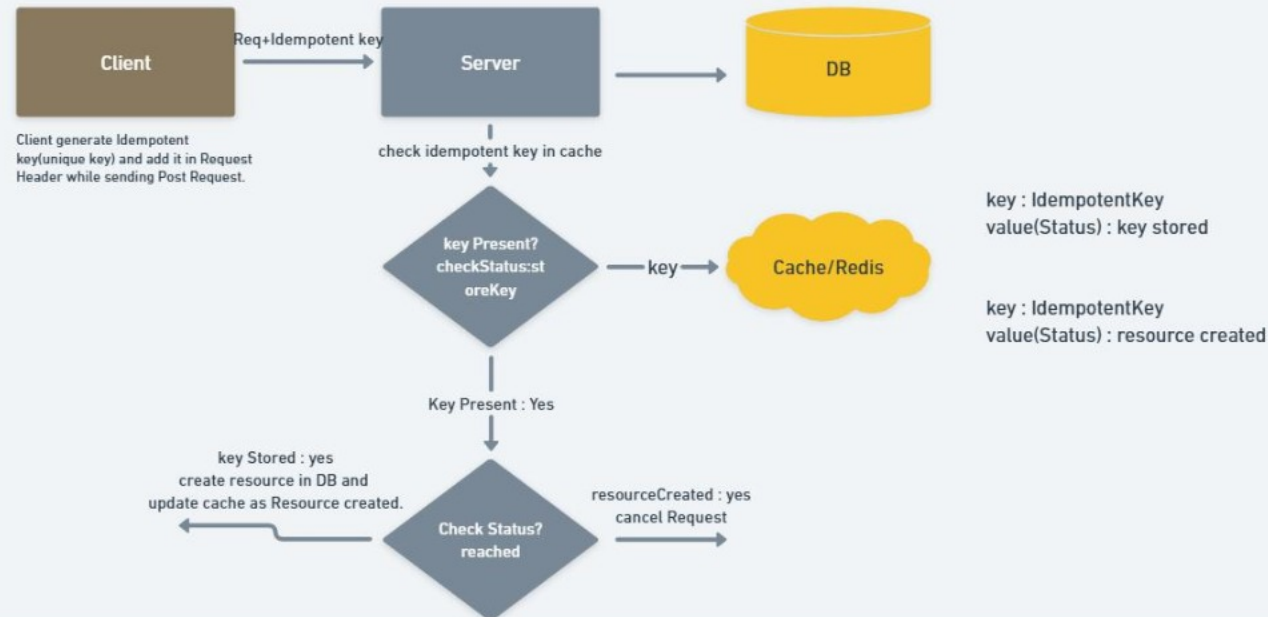
Service - Service

Functional Requirements :

- send Request
- check Request
- accept/reject Request

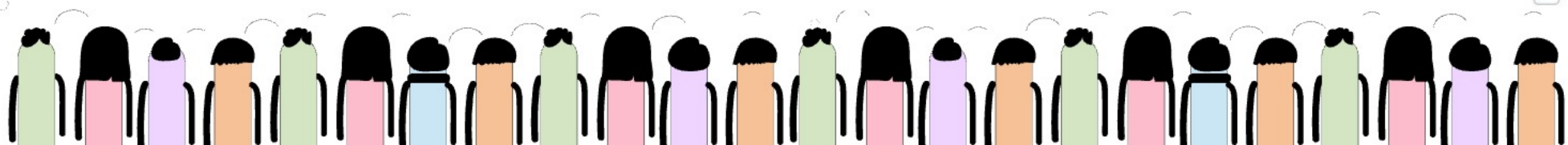
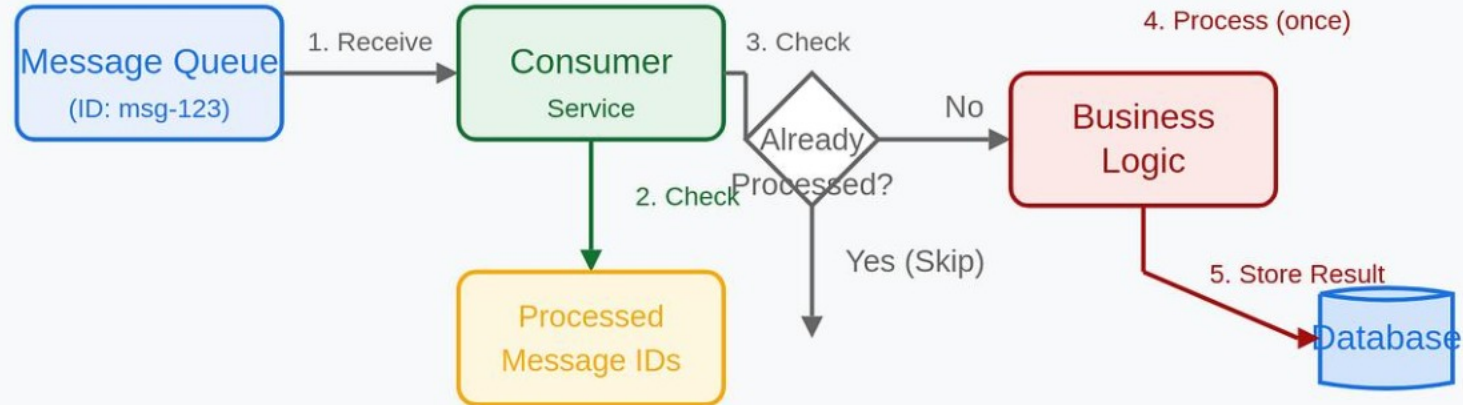
Non-Functional Requirements:

- Highly Consistent
- Latency : 300 ms

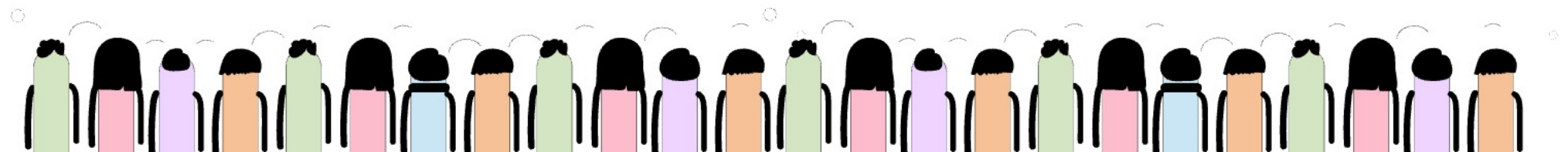
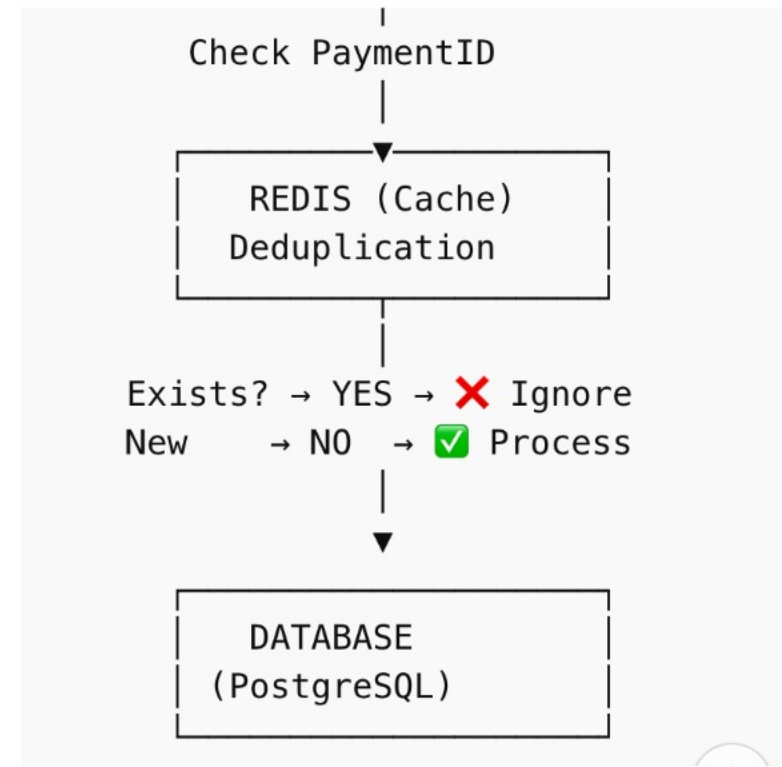
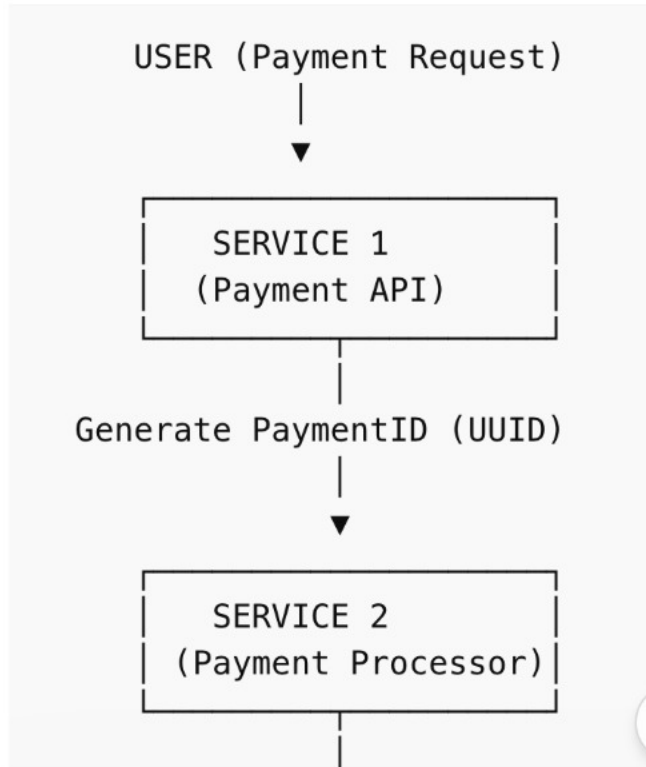


Service - Service

Idempotent Consumer Pattern



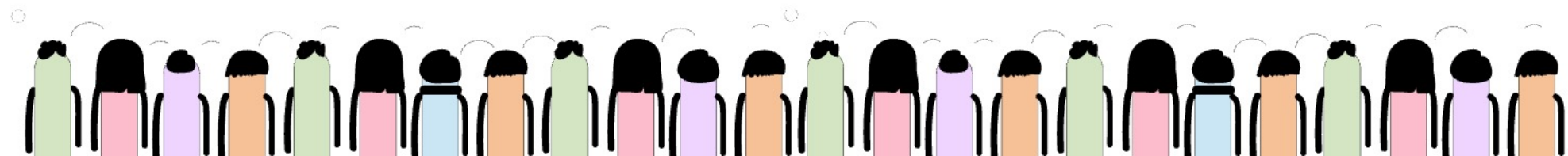
Service - Service



Service - Service

FLOW (SUPER SIMPLE)

1. User sends payment
2. Service1 creates PaymentID
3. Service2 receives request
4. Redis check:
 - Already processed → ignore
 - New → process payment
5. Store result in DB



Service - Service

🔥 Common Real-World Usage

💳 Payments

- Stripe, PayPal
→ Prevent double charges on retry

🛒 E-commerce Orders

- Amazon
→ Prevent duplicate orders

🚗 Ride Booking

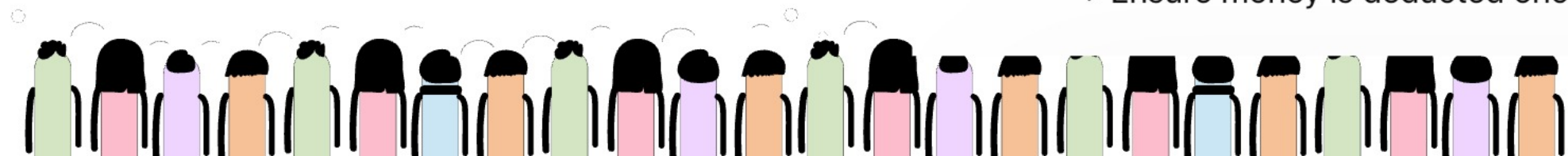
- Uber
→ Prevent duplicate ride creation

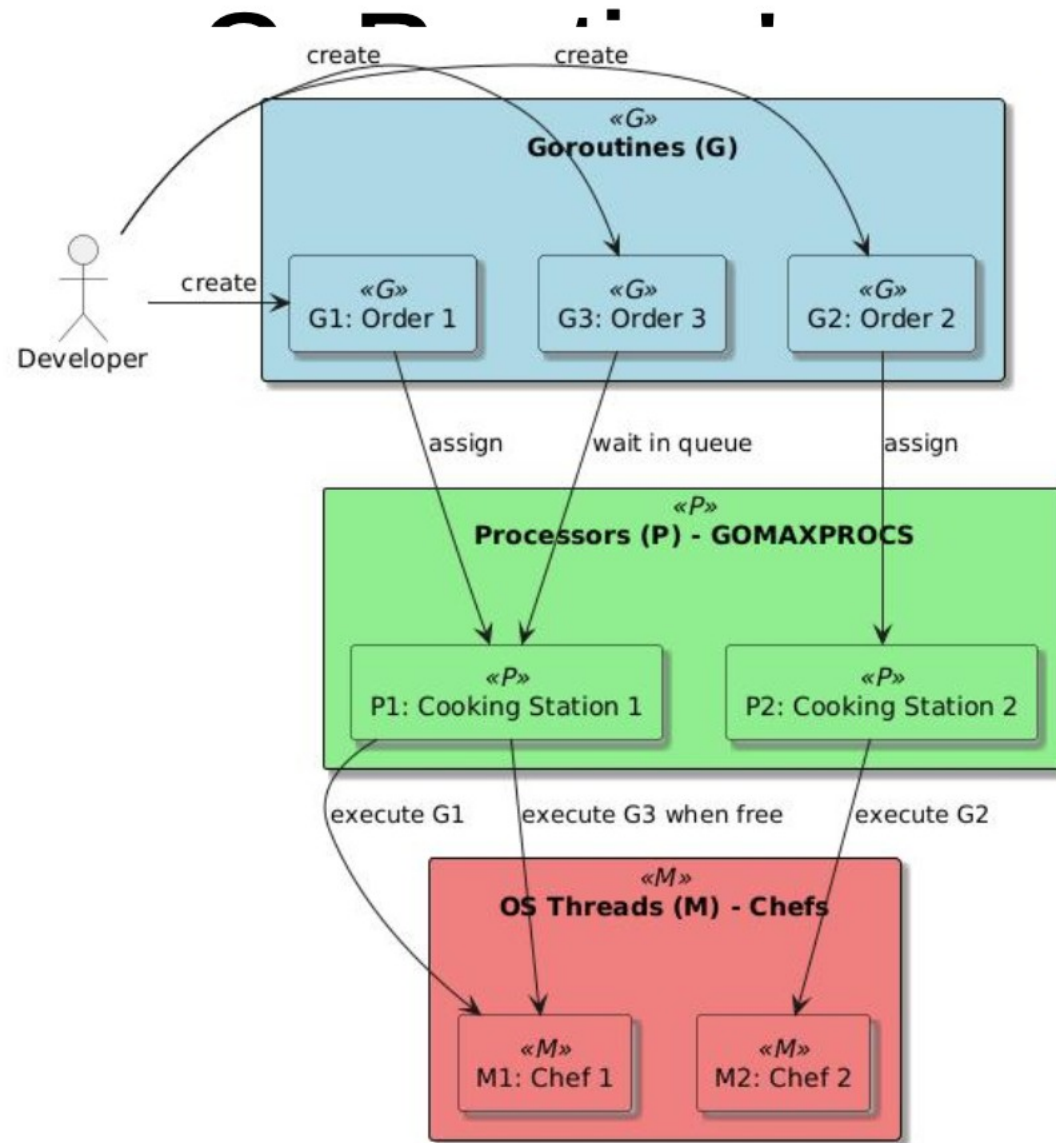
📡 Event Streaming

- Apache Kafka
→ Prevent duplicate event processing

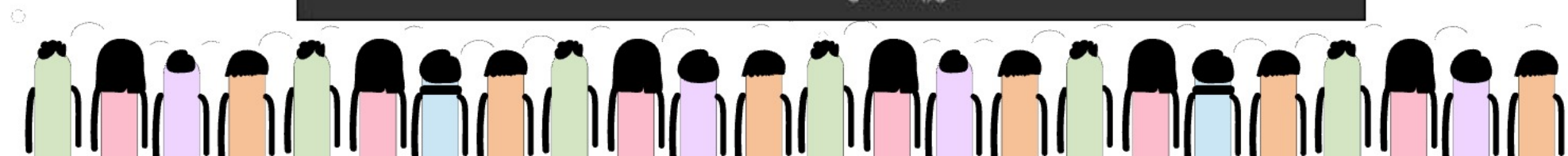
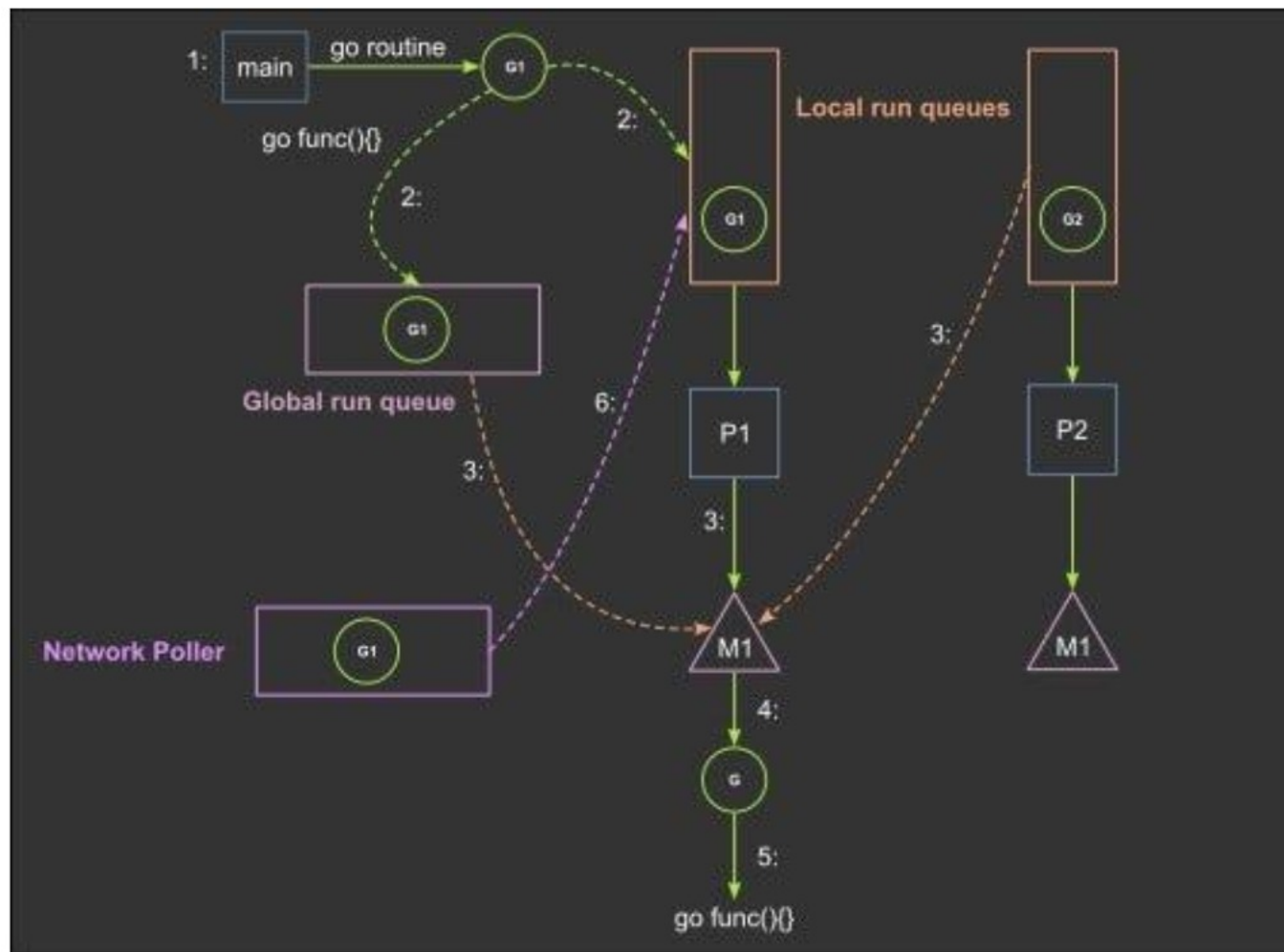
🏦 Banking Transactions

- Visa
→ Ensure money is deducted once

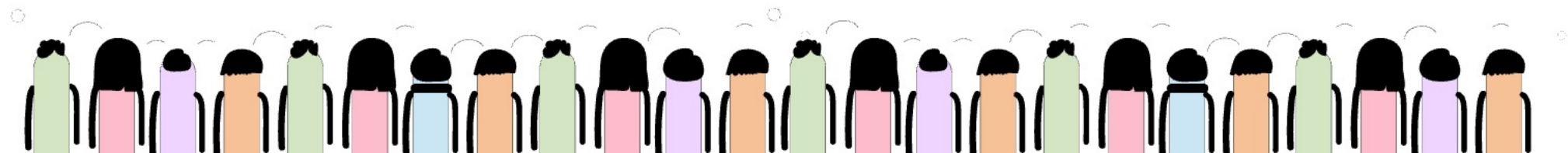
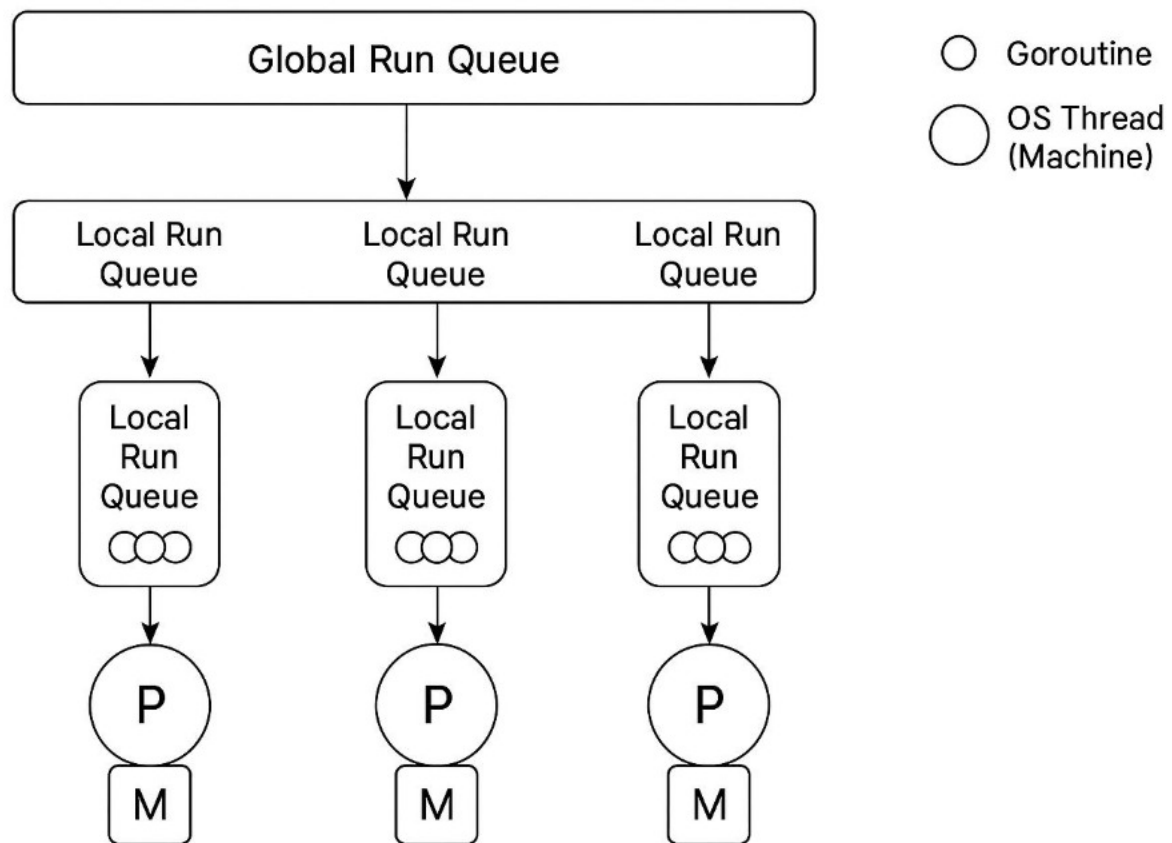




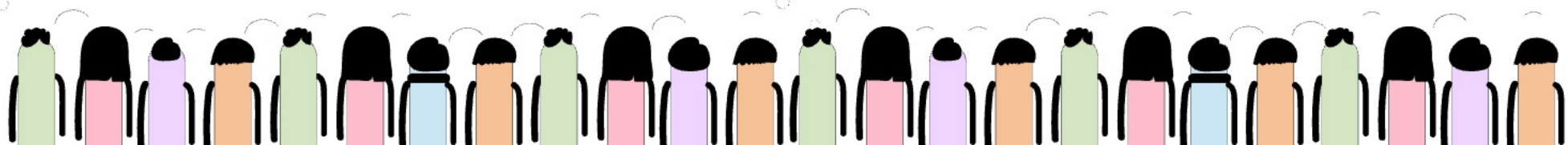
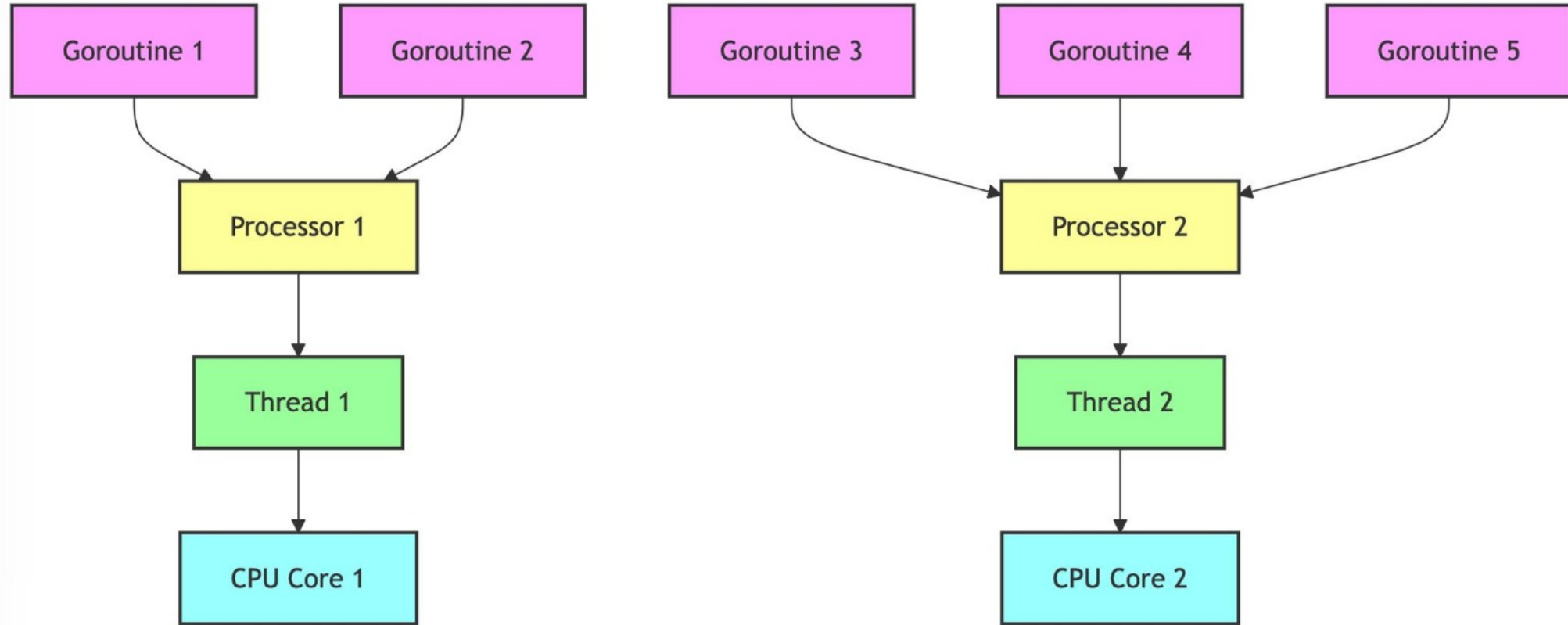
GoRoutine's



GoRoutine's



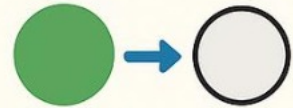
GoRoutine's



GoRoutine's

Quick Recap: From Channels to Scheduler

- 1** A goroutine blocks on a channel



`msg := <-ch`



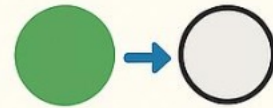
'Red light' – must wait

The scheduler records:
"This goroutine is paused –
don't waste CPU on it."

- 3** The scheduler swaps goroutines on OS threads

Just like traffic lights coordinate
multiple intersections

- 2** A complementary goroutine arrives



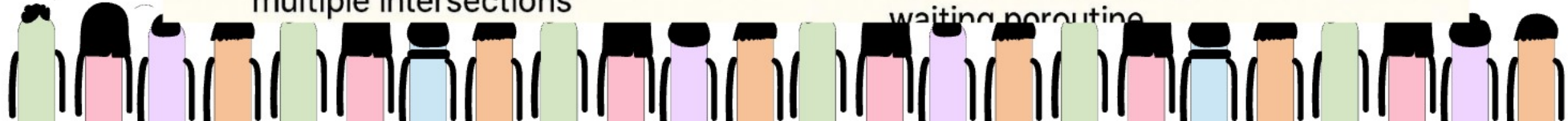
`ch <- "go!"`



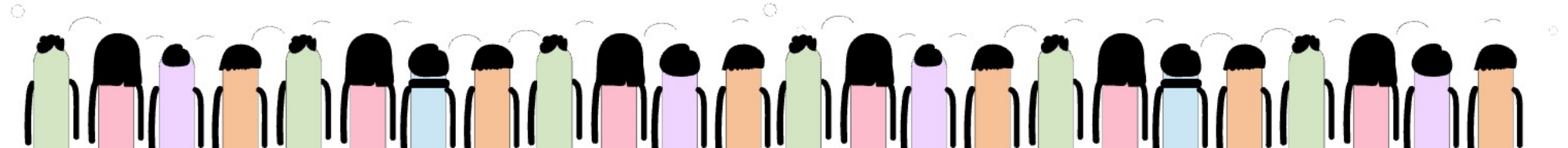
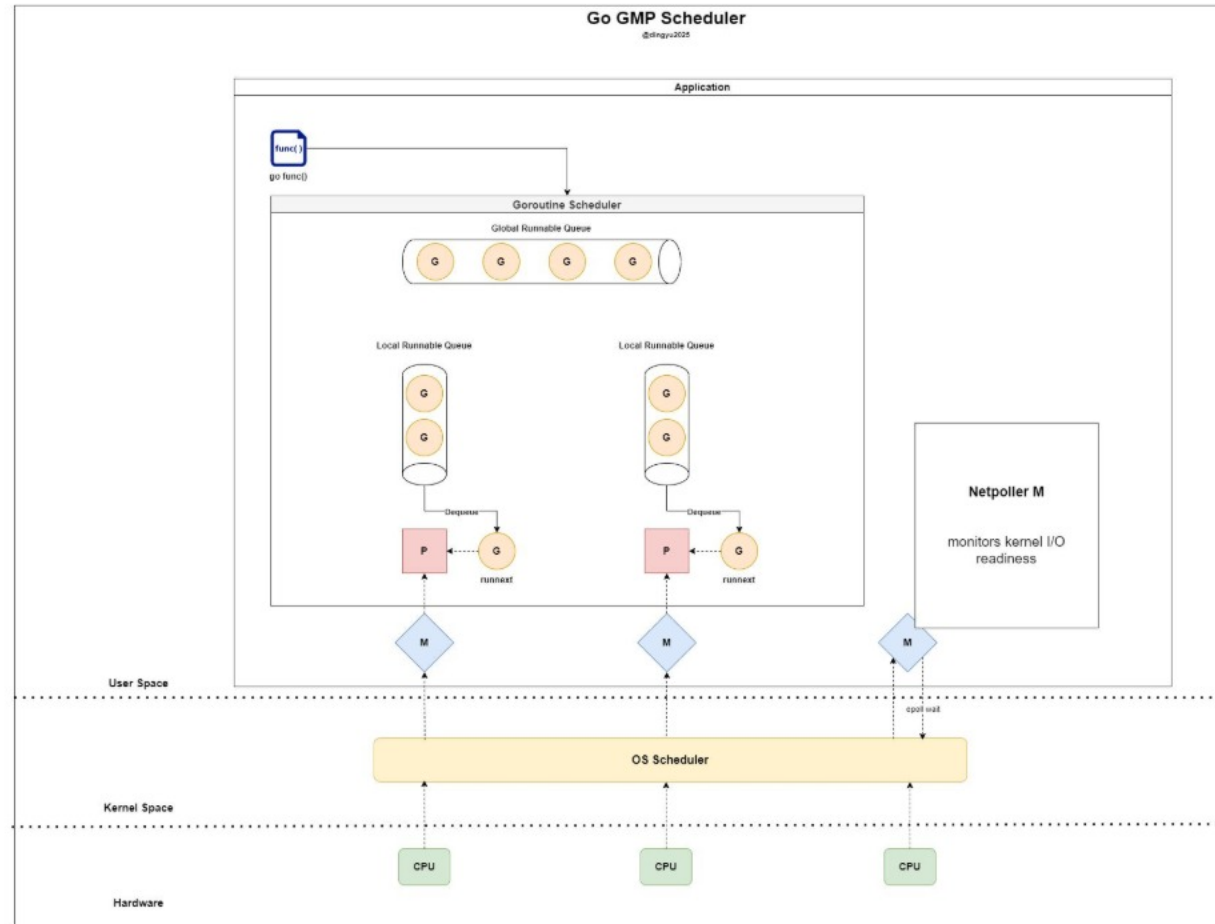
Now the scheduler says:
"Wake up that waiting goroutine –
it can continue."

- 4** Work stealing keeps CPUs busy and balanced

If one thread is idle and another busy,
scheduler steals the
waiting goroutine



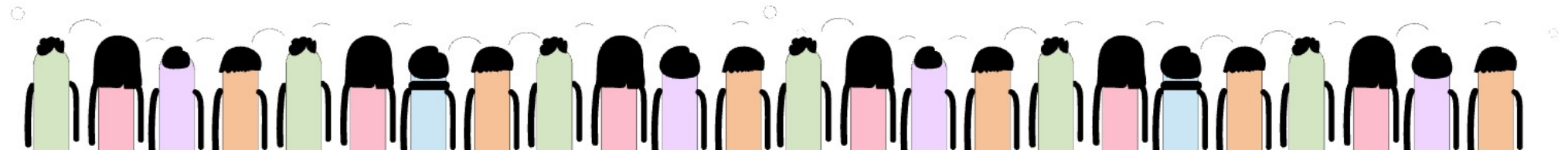
GoRoutine's



GoRoutine's

Internal Flow

1. You create 200+ goroutines
2. Each goroutine (G) goes into run queue
3. Each P has local queue
4. Scheduler:
 - picks goroutine
 - assigns to thread (M)
5. If one P is idle → steals work from another P



GoRoutine's

✓ SCENARIO 1: CPU-BOUND WORK

Problem:

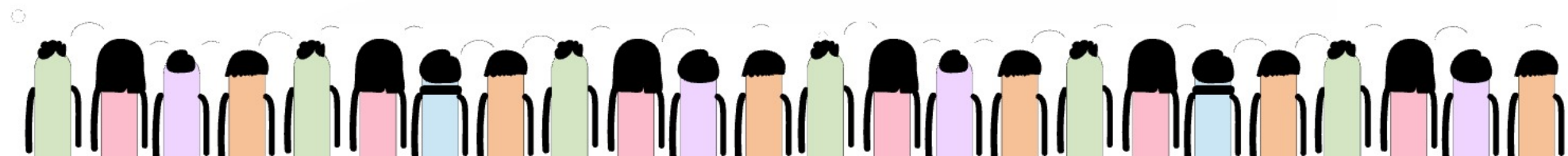
Too many goroutines fighting CPU

Fix:

```
</> Go  
  
workerCount := runtime.NumCPU()
```

👉 Rule:

CPU-bound → workers ≈ CPU cores



GoRoutine's

✓ SCENARIO 3: HIGH LOAD (1M+ JOBS)

Problem:

Burst traffic → queue overload → latency spike

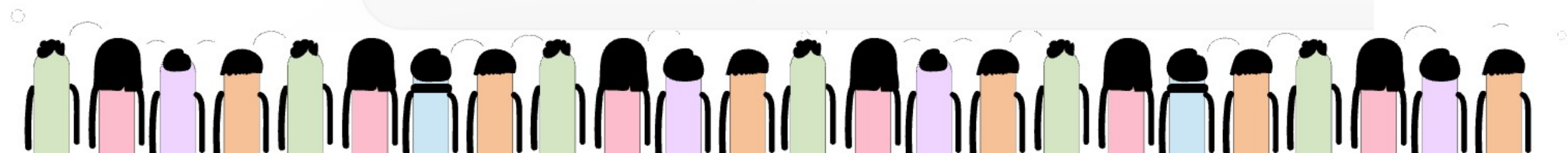
Fix: Worker Pool + Backpressure

⌘ Go

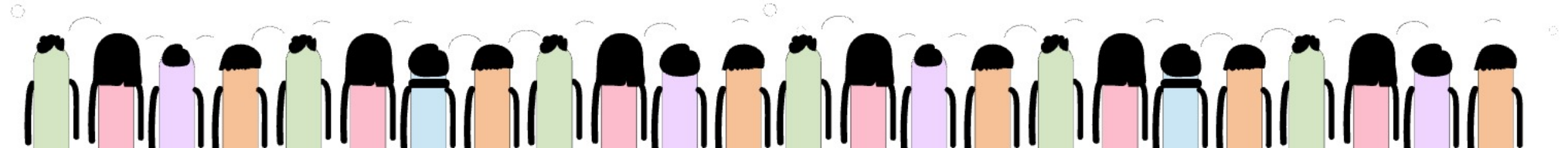
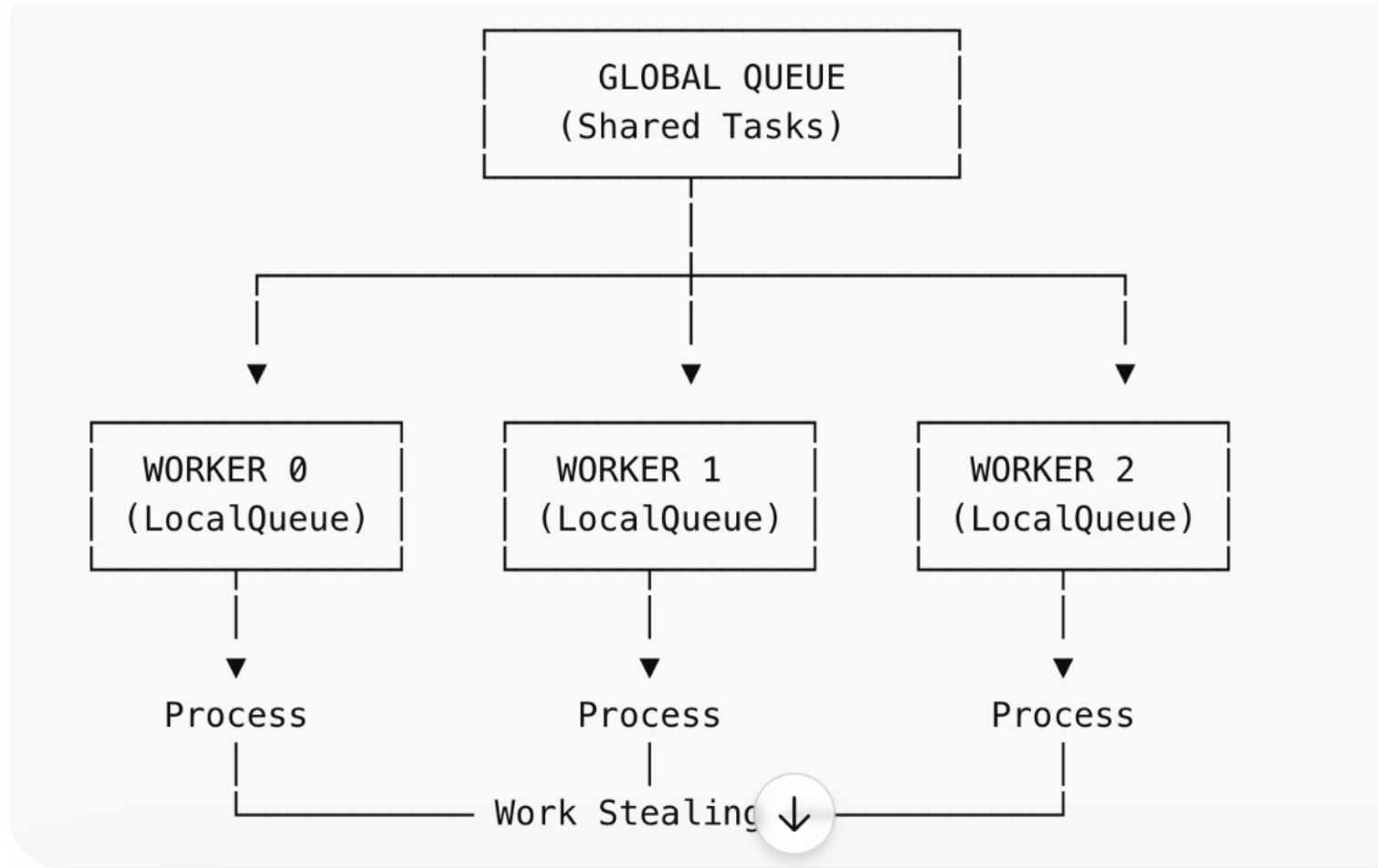
```
jobs := make(chan Job, 10000) // bounded
```

👉 If full:

Producer blocks → system stabilizes



GoRoutine's CREATION



GoRoutine's

START

Initialize:

- GlobalQueue
- LocalQueues[0..3]

Producer:

→ Push all tasks into LocalQueue[0]

Start Workers (goroutines)

Worker Loop:

Step 1: Try Local Queue

- └ If task found → process
- └ Else →

Step 2: Try Global Queue

- └ If task found → process
- └ Else →

Step 3: Work Stealing

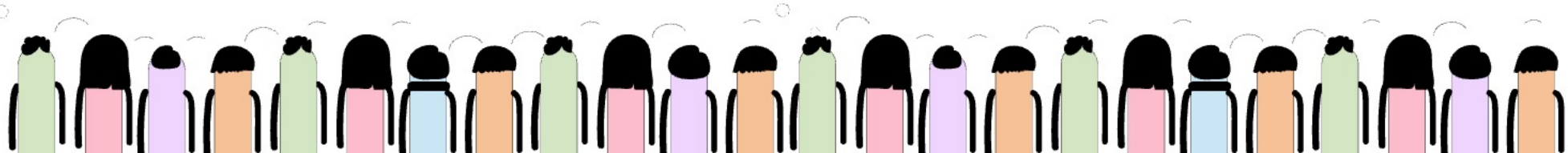
- └ Pick another worker
- └ Steal half tasks
- └ Push into local queue
- └ Continue loop

Step 4: No work anywhere

→ Exit worker

WaitGroup waits

END (All tasks completed)



GoRoutine's

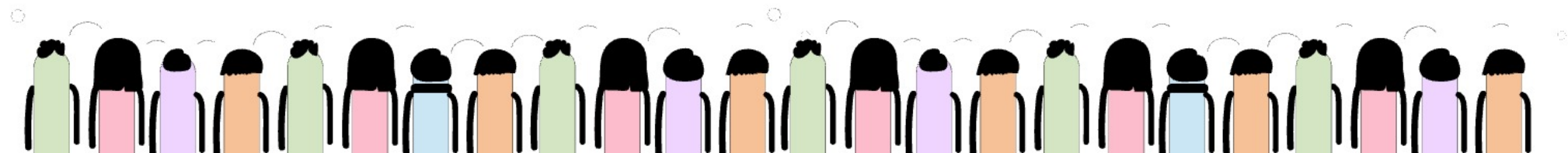
◆ MAIN()

1. Create 4 workers
2. Create 4 local queues
3. Put ALL tasks into worker 0 queue
4. Start goroutines (workers)
5. Wait until all finish

◆ WORKER()

LOOP:

1. `local.Pop()`
→ fast path
2. `global.Pop()`
→ fallback
3. steal from others
→ load balancing
4. if nothing found
→ exit



GoRoutine's

◆ STEAL LOGIC

Worker A idle
↓
Check Worker B queue
↓
Take HALF tasks
↓
Push into own queue
↓
Start processing

Your Code

LocalQueue

GlobalQueue

worker()

Task

StealHalf

Go Runtime

P.runq

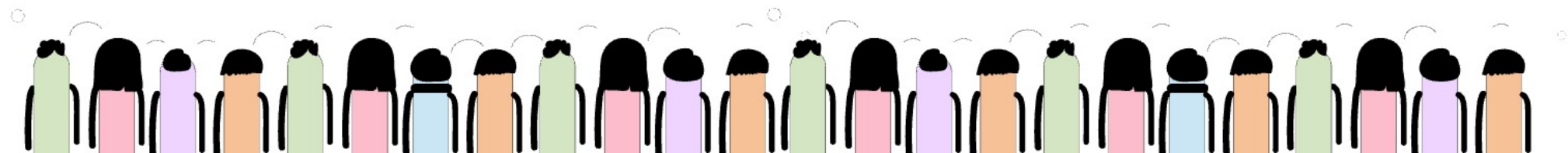
global runq

M (thread)

Goroutine (G)



Work stealing

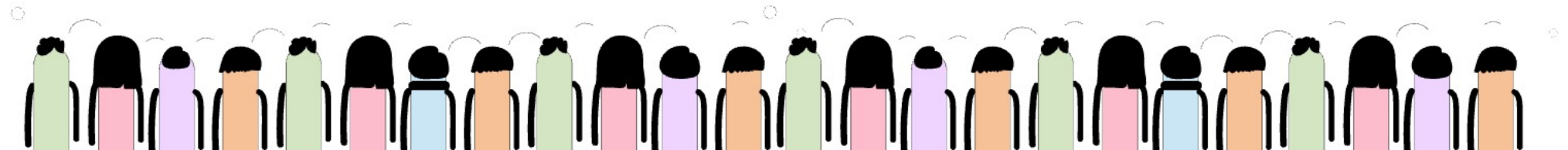


GoRoutine's

1) G-M-P model (not just threads)

- Goroutines (**G**) are much lighter than OS threads.
- **P (processor)** is a scheduling context that owns a local run queue.
- **M (thread)** executes Gs but only when it has a P.

👉 This separation lets Go run **millions of goroutines** efficiently.



GoRoutine's

2) Per-P local queues (cache-friendly scheduling)

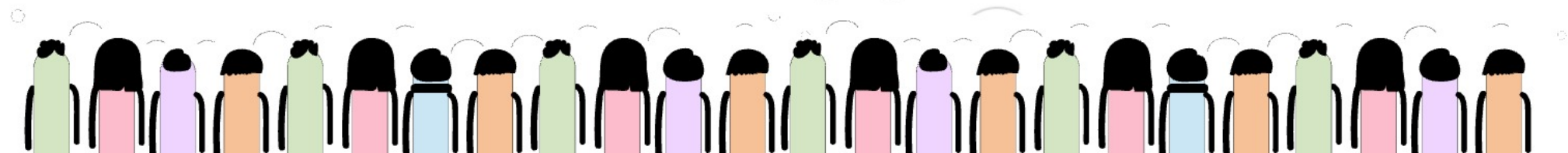
- Most scheduling happens **locally** (no global lock).
- This reduces contention and improves CPU cache locality.

👉 Big performance win compared to a single global queue.

3) Work stealing (automatic load balancing)

- Idle P steals work from busy P.
- Keeps all CPUs busy without central coordination.

👉 Eliminates manual load balancing logic.



GoRoutine's

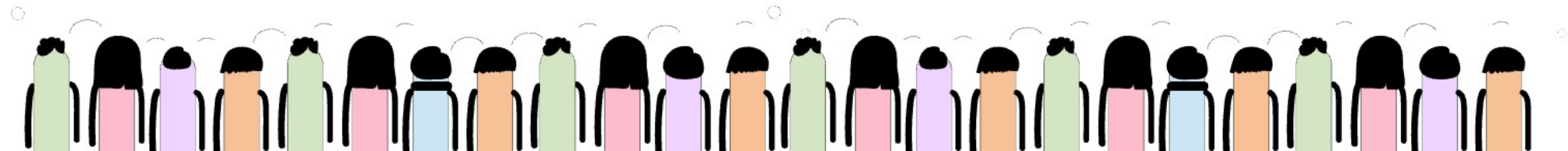
4) Cheap goroutines (stack + growth model)

- Start with tiny stacks (~2 KB) that grow/shrink.
 - Enables huge concurrency without memory blow-up.
-

5) Parking / unparking (non-blocking scheduler)

- Blocking operations (channels, syscalls) don't block threads.
- Goroutines are **parked** and threads are reused.

👉 This is critical for high throughput.



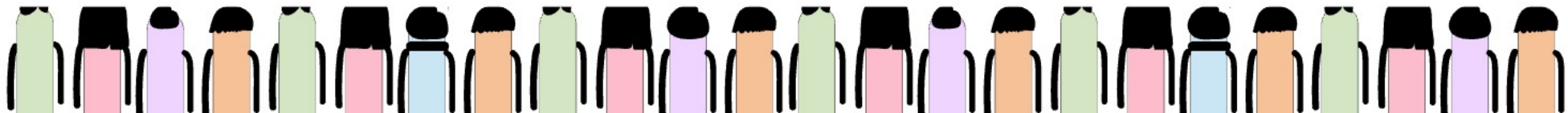
GoRoutine's

6) Integrated network poller (netpoll)

- IO waits are handled efficiently without blocking threads.
 - Scheduler wakes goroutines when IO is ready.
-

7) Concurrent garbage collector

- Low pause times while goroutines keep running.
- Designed specifically for highly concurrent workloads.

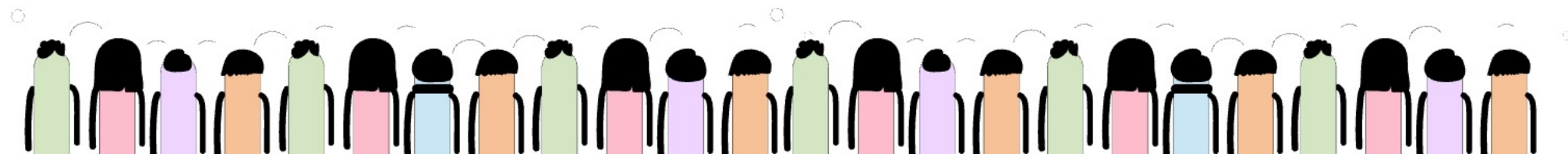


Go Lang Creation

🧠 Innovation Weighting (What really matters)

📊 Weighted Breakdown

#	Concept	Impact	Why it matters
1	G-M-P Model	★★★★★ (25%)	Core foundation of Go scheduler
2	Local Queues (per-P)	★★★★★ (15%)	Removes contention, improves cache locality
3	Work Stealing	★★★★★ (15%)	Automatic load balancing
4	Cheap Goroutines	★★★★★ (20%)	Enables massive concurrency

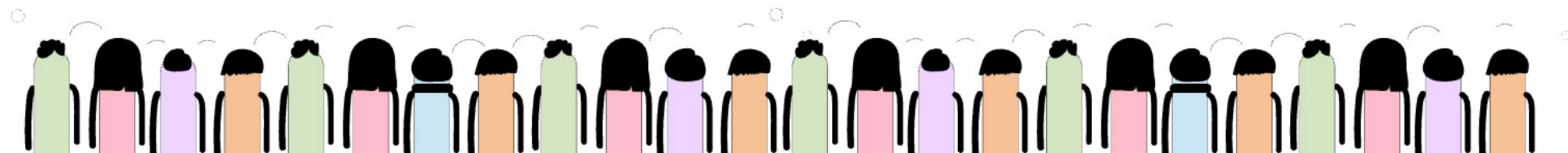




Go Lang Creation



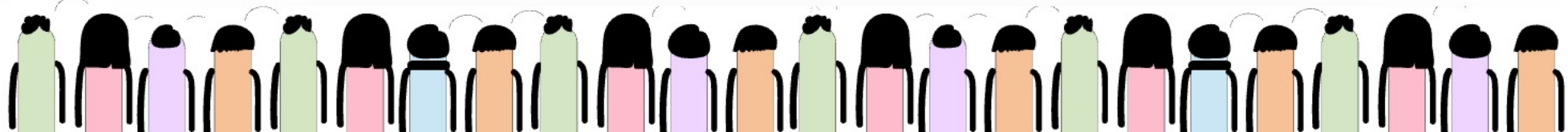
5	Parking / Unparking	★★★★ (10%)	Efficient blocking without wasting threads
6	Netpoll (IO integration)	★★★★ (10%)	Critical for real-world systems
7	Concurrent GC	★★★★ (5%)	Stability + low latency



Go Lang Creation

Full Language Creation – Weighted Breakdown

Layer	Weight	Why it matters
Runtime (scheduler, GC, memory)	★★★★★ 30%	Hardest + performance-critical
Compiler / Interpreter	★★★★★ 25%	Converts code → executable
Type System	★★★★★ 15%	Safety, correctness, developer experience
Standard Library	★★★★★ 15%	Real usability of the language
Syntax / Parser	★★★★ 10%	User-facing, but easier technically
Tooling (build, debug, pkg mgmt)	★★★ 5% 	Ecosystem maturity





Go Lang Comparsion



Layer	Go	Java	Rust	C++	Python
Runtime	30%	35%	10%	5%	40%
Compiler	25%	20%	35%	40%	10%
Type System	15%	20%	30%	25%	5%
Std Library	15%	15%	10%	10%	20%
Syntax	10%	5%	10%	15%	15%
Tooling	5%	5%	5%	5%	10%

